

Atividade Prática – Análise de Relato ***Engenharia de Requisitos no Desenvolvimento de Software***

Objetivo

Compreender quais fatores de fracasso e sucesso de projetos estão relacionados com uma abordagem correta de Engenharia de Requisitos

Instruções: Ler material indicado e responder as questões.

Introdução

O relato a ser analisado são cenários elaborado por um profissional de larga experiência na área. O relato é uma anedota e exagera o cenário de fracasso de projetos comumente encontrado no mercado. Captar os fatos e eventos do cotidiano e relacionar a boas práticas e metodologias conhecidas permite ao profissional identificar soluções adequadas.

Tarefa 1: Leia o relato

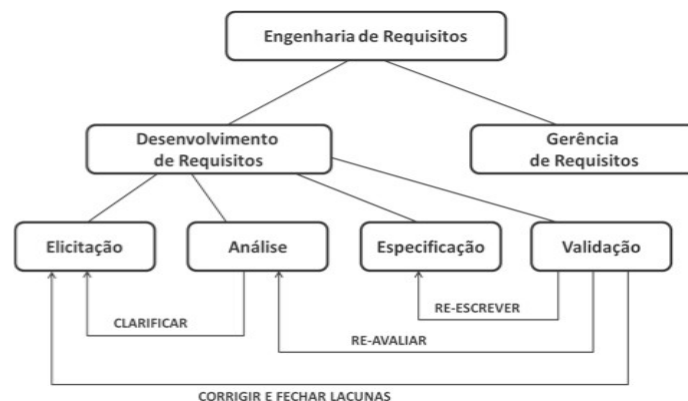
No artigo “O Ciclo Vicioso do Software Retranqueiro”, Klaus Wuestfeld apresenta um cenário caricato (porém real) de como organizações e profissionais podem entrar em um ciclo improdutivo e dispendioso de desenvolvimento de software, que se torna cada vez mais burocrático pelo próprio insucesso de ações anteriores.

Tarefa 2: Revise os conceitos

No livro : Capítulo 7 (Engenharia de Requisitos), “Engenharia de Software” de Ian Sommerville.

Tarefa 3: Responda as questões

- 1) Quais pontos do relato de Klaus Wuestfeld estão diretamente relacionados com a área de Requisitos?
- 2) Quais atividades você sugere que sejam realizadas para evitar os problemas que podem vir acontecer em um projeto (diretamente relacionados com a área de Requisitos)? (Utilize as atividades da Engenharia de Requisitos para lhe guiar/ajudar nas sugestões de atividades).



ANEXO

O Círculo Vicioso do Software Retranqueiro por Klaus Wuestefeld

Projetos de desenvolvimento de software fracassam. Fracassam notoriamente e fracassam feio. De acordo com o Gartner Group, 80% a 90% dos projetos de TI fracassam, sendo cancelados ou entregues com prazo ou com o orçamento estourados.

Mas qual é a importância disso, afinal? Por que as organizações dependem tanto do desenvolvimento de software próprio? Por que não podem simplesmente comprar todo seu software pronto?

É claro que não estamos falando de commodities como sistemas operacionais ou software para automação de escritório. Assim como a escolha da marca de clips, cada vez menos importa qual editor de textos ou qual ambiente gráfico vai-se usar.

A maioria das empresas depende também dos famosos "pacotões": folha de pagamento, controle de estoque, contas a pagar, etc. Embora eles ainda tragam muitos problemas de implantação e integração, principalmente em grandes organizações, também não é a escolha do "pacotão" que vai GARANTIR vantagem competitiva a essa ou aquela empresa.

Onde é que a coisa pega, então? Qual é o tipo de software que realmente faz a diferença?

Organizações como restaurantes, grupos de teatro, clínicas médicas e bandas musicais não ligam muito para software. No caso delas, o que importa mesmo é a experiência, o conhecimento tácito de seus integrantes.

Para grande maioria das empresas, porém, informação e conhecimento explícito estão, cada vez mais, definindo o resultado do jogo. É justamente nos sistemas centrais dessas organizações, portanto, que agilidade e qualidade no desenvolvimento de software fazem a diferença entre o sucesso e a morte.

Software é a única coisa que consegue representar conhecimento explícito e, ao mesmo tempo, botá-lo para trabalhar em favor da empresa, em vez de ficar fazendo volume numa prateleira ou num lindo repositório de documentos.

Software é conhecimento explícito por excelência.

Jamais conseguirá uma empresa superar a outra, se comprar exatamente o mesmo software de prateleira para seu sistema central. O máximo que conseguirá atingir com isso é a mediocridade e, não sendo capaz de alterar esse mesmo software para capturar seus conhecimentos adquiridos frente ao dinamismo do mercado, ficar estagnada é o que lhe resta.

Mas por que isso acontece? Por que, mesmo sendo cada vez mais importantes para as organizações, os projetos de software continuam fracassando? Quais são as causas dessa situação vergonhosa para a minha profissão, como desenvolvedor de software?

Não existe causa única. Existe um círculo vicioso de causas que se exacerbam mutuamente.

- 1) O fracasso da grande maioria dos projetos de desenvolvimento de software causa grandes prejuízos aos clientes.
- 2) Os repetidos prejuízos alimentam a desconfiança dos clientes em relação ao desenvolvimento e aos desenvolvedores de software.
- 3) A desconfiança faz com que os clientes peçam orçamentos de preço fechado e, em muitos casos, com pesadas multas.
- 4) Para fazer orçamentos de preço fechado, os desenvolvedores têm que fazer estimativas precisas.

Requisitos de Software

- 5) Para fazer estimativas precisas, os desenvolvedores necessitam de requisitos muito bem definidos.
- 6) Para obter requisitos muito bem definidos, cliente e desenvolvedores passam boa parte do tempo de um projeto, mesmo antes de seu início, levantando e detalhando requisitos. Trata-se de um investimento pesado para um projeto que nem se sabe ao certo se vai ser implementado.
- 7) Para minimizar esse investimento, que muitas vezes não é pago pelo cliente, os desenvolvedores comprometem a qualidade e abrangência do levantamento de requisitos (ver 10).
- 8) Depois de trabalhar tanto na produção de um lindo documento de requisitos, o cliente não quer mais saber de falar com o desenvolvedor até a entrega do sistema final.
- 9) O desenvolvedor, por sua vez, dá graças por não ter que aturar o cliente que só serve para atrapalhar, mudar de idéia e perguntar como anda o projeto.
- 10) Evitando o contato com o cliente e com pressa, o desenvolvedor passa a supor uma série de coisas a respeito de como o sistema deveria funcionar. Por mais pesado que tenha sido o investimento na fase de requisitos, eles sempre apresentam inconsistências, ambigüidades e omissões. Se não fosse assim, bastaria compilar o documento de requisitos e o sistema estaria pronto.
- 11) A maioria das suposições dos desenvolvedores está correta. A minoria, porém, está errada e causa surpresas desagradáveis para o cliente (ver 25).
- 12) As surpresas desagradáveis causam retrabalho (ver 24) para o desenvolvedor quando da entrega do produto.
- 13) Com pressa, a cada funcionalidade implementada, o desenvolvedor faz apenas alguns testezinhos manuais básicos. Ele não tem tempo para escrever código de teste automatizado para testar seu sistema. A qualidade do software fica prejudicada (ver 23).
- 14) O desenvolvedor é proibido de melhorar código que já está funcionando para eliminar as duplicações com novas funcionalidades. "Se tá funcionando, então não mexe!". A complexidade do sistema passa a crescer como uma colcha de retalhos com duplicações, dependências a torto e direito e acoplamento forte entre os módulos do sistema.
- 15) A complexidade faz as modificações no sistema serem cada vez mais demoradas e propensas a erro (ver 23).
- 16) A demora e o alto custo de modificações atrasam o projeto (ver 10, 13, 21, 25).
- 17) Os atrasos causam os ciclos de desenvolvimento a serem cada vez maiores.
- 18) Quanto maior o tempo de cada ciclo de desenvolvimento, piores as surpresas, menos freqüente a chance do cliente requisitar novas funcionalidades e maior o tempo de espera.
- 19) Quanto maior o tempo de espera, mais funcionalidades o cliente tem que requisitar nas raras chances que consegue: "Quero tudo, o mais configurável, flexível e genérico possível."
- 20) Exagero de requisitos torna o software mais complexo (ver 15) e aumenta o tempo de desenvolvimento (ver 16).
- 21) Projetos atrasados aceleram ou simplesmente ignoram a fase de testes.
- 22) Sem a fase de testes a qualidade que já estava ruim vai pro buraco.
- 23) Os defeitos causam retrabalho.
- 24) O retrabalho causa atraso (ver 10, 13, 21, 17) e ainda mais defeitos (ver 23).
- 25) As surpresas desagradáveis, os defeitos e os atrasos determinam o fracasso do projeto (ver 1).

Desenvolvedores experientes podem facilmente duplicar o tamanho dessa lista de sintomas do círculo vicioso. Todas essas coisas são tão corriqueiras na vida dos desenvolvedores que a grande maioria encara

essa situação lastimável como sendo normal, inevitável.

A única forma de um projeto complexo nessas condições ser entregue dentro do prazo e dentro do orçamento é com pessoas excepcionalmente talentosas, experientes e motivadas, ou simplesmente com um prazo e com orçamento absurdamente altos.

Quando não estão preocupados com diagramas mais imponentes, cronogramas mais vistosos e coisas do gênero, os processos de desenvolvimento tradicionais trabalham alguns dos sintomas do círculo vicioso, buscando técnicas melhores para conseguir: levantamento de requisitos mais completos, estimativas mais precisas, modelos mais rastreáveis, etc.

Infelizmente, como em qualquer círculo vicioso, atacar os sintomas é perda de tempo, paliativo no melhor caso. Ainda por cima, para a maioria dos projetos doentes, o mero peso burocrático dos processos tradicionais, por si só, já representa uma quimioterapia por demais brutal. Não é à toa, portanto, que cansamos de ver até desenvolvedores sérios e responsáveis largando o processo, a quimioterapia, e preferindo encarar na raça a doença.

Esses desenvolvedores, cujo talento, como já vimos, é a única chance de sucesso desses projetos, ainda são condenados pelos feitores nas "fábricas de software" (triste metáfora), por não estarem atuando como deveriam: como "máquinas" burras seguindo o processo pseudo-definido.

A influência mais perniciosa nesse contexto todo, porém, ainda são as empresas fornecedoras de ferramentas para desenvolvimento de software e organizações certificadoras de processos. Para vender seu peixe, elas têm interesse em que o processo de desenvolvimento adotado pelo mercado seja o mais complexo e burocrático possível.

Mas será que esse círculo vicioso é realmente inevitável? Será ele um fenômeno da natureza? Ou será que podemos acabar com isso?

Os sucessos alcançados por equipes ágeis vem causando comoção justamente neste mercado cansado das metodologias retranqueiras tradicionais e do notório fracasso de seus projetos. O maior evento nacional de engenharia de software, até 2010, foi o Agile Brasil, onde tive a oportunidade de fazer a palestra de encerramento.

XP e as demais metodologias ágeis simplesmente elevam a um novo patamar o significado de "qualidade" e "agilidade" em desenvolvimento de software.

Sobre o autor:

Pioneiro de Extreme Programming (XP) no Brasil, Klaus Wuestefeld é autor do manifesto da computação soberana e do Prevayler, a camada de prevalência de objetos original. Klaus é sócio-fundador da Objective Solutions e já atuou como desenvolvedor, líder de projeto, gerente e até como cliente em projetos usando XP.

Copyright 2002-2012 Klaus Wuestefeld. Todos os direitos reservados.

Autorizo a publicação deste artigo na íntegra em qualquer meio, incluindo este aviso de copyright.

Twitter: [@KlausWuestefeld](https://twitter.com/KlausWuestefeld)